

个人主页开发报告

Personal Homepage — Development Report

Bokang Liu (刘博康)

计算机科学与技术 · 2026 级

线上地址

<https://carrotsoup123456.github.io/homepage/>



2026 年 10 月 7 日

目录

一、开发历史	V1 起步 → V2 多功能主页 → V3 视觉重设计 → V4 反馈迭代，附版本截图与 Git 记录
二、新增或修改的主要功能	每个功能做了什么，以及想法从哪里来（真实访客反馈 / 自主优化动机）
三、遇到并解决的问题	线上崩溃、游戏文字溢出、动效过度、与 AI 协作的流程事故——原因与修法
四、收集到的反馈与决策过程	6 条真实访客反馈的完整决策表：采纳的理由、不采纳的理由
五、简短反思	这门课教会我的事

一、开发历史

整个主页在 2026 年 10 月 3 日至 7 日的五天里完成了四个大版本，全部开发过程与 AI 结对完成，代码与文档全程用 Git 管理（main 分支 105 次提交），每一步都有对话记录和测试证据留痕。

1.1 V1 起步：先有一个能用的主页（10 月 3 日）

第一版是单页简历：英雄区（头像、姓名、简介）、技能、项目、经历、教育、联系方式，六大区块一页排完。想法很朴素——**先有再好**：先把"我是谁"完整地放到线上，再谈花样。技术栈选了 Vue 3 + Vite，从第一天起就坚持每轮改动跑自动化测试。



图 1-1 · V1 首页：单页简历，白底蓝调

1.2 V2 多功能主页：从一页纸到一个小站（10 月 3 日）

一天之内把单页扩成了多路由小站：关于页、知识库（初版"建设中"）、项目详情页、联系页。最有价值的突破是联系表单接上了真实后端（FormSubmit 邮件转发）——填完真的会收到邮件，这让主页第一次有了"活着"的感觉。



图 1-2 · V2：多路由站点（左）；联系表单真实提交成功，邮件送达（右）

1.3 V3 视觉重设计：给主页一个灵魂（10 月 4 日）

前两版是“能用但普通”。V3 做了一次彻底的视觉重设计，主题定为「胡萝卜园」（呼应我的网名 carrotsoup）：暖白纸感底色、深浅双模式、衬线大标题、四处动态效果（首屏萤火虫、数字滚动、鼠标柔光、reduced-motion 降级）。随后又上线了**页内反馈组件**——访客一键就能给我发“这个页面有用/没用+一句话建议”，这正是后来 V4 迭代的弹药来源。知识库也从“建设中”变成了真实笔记内容（构建时从 Markdown 文件自动解析）。

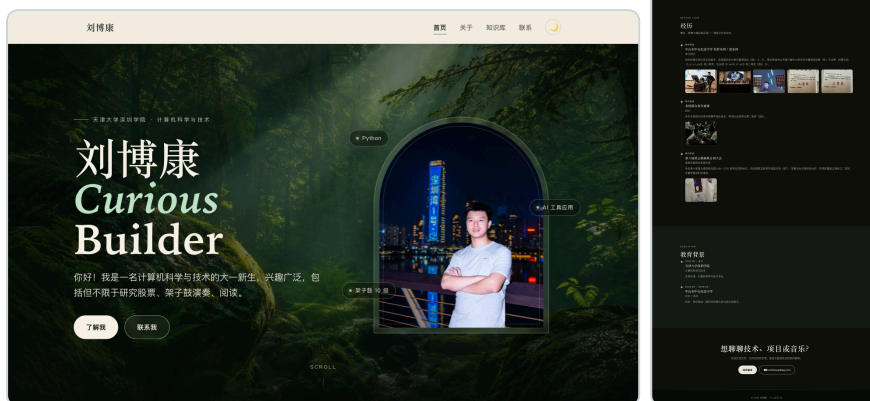


图 1-3 · V3：胡萝卜园主题，浅色（左）与深色（右）



图 1-4 · V3 上线的页内反馈组件：三级流程（询问→补充说明→已提交），提交直达我的邮箱

1.4 V4 从反馈到产品迭代（10 月 4 日–7 日）

V4 是本次作业的核心：我把主页链接发到朋友圈和同学群，收集到 6 条真实访客反馈，再按照"分类 → 定优先级 → 修复 → 线上验证 → 回归测试"的完整流程全部处理完。修复内容涵盖：亮点前置导航、动效一收一放、兴趣四宫格、技术路线流程图、剧情游戏手机适配。截至提交日，站点运行 v6.40 版本，自动化测试 114 条全绿，六项线上回归全部通过。详细决策过程见本报告第四章。

版本控制：全部代码托管在 GitHub（main 分支 40+ 次提交，gh-pages 分支自动部署）。每次功能改动都单独成提交，配合对话记录可完整回溯"谁在什么时间为什么改了什么"。

二、新增或修改的主要功能

这一章按"功能 → 想法来源"组织。你会发现一半以上的功能直接来自真实反馈——这是我有意为之：让访客的声音决定产品的方向，而不是自嗨式堆功能。

2.1 亮点前置 + 分节目录跳转（V4 · 来自反馈 #1）

访客 A 反馈："内容很多但不知道先看哪块。" 我把每篇笔记最有价值的 2–3 个亮点提到正文最前面做成卡片，并在顶部加了分节目录条，点击直接跳到对应章节。



图 2-1 · 反馈 #1 前后对比：亮点原来沉在页尾（左），现在第一屏就能看到（右）

2.2 动效"一收一放"（V4 · 来自反馈 #2 + #5）

一位访客说"动效有点花"，另一位却说"图片展示可以加点特效"——两条反馈方向相反，正好逼我想清楚动效的定位：背景动效收敛（删除全部萤火虫、漂浮动画放慢一倍、滚动显现幅度减半），前景交互增强（桌面端悬停卡片上浮 + 图片微放大）。一收一放之后，页面安静了，但互动更精致了。

2.3 兴趣四宫格（V4 · 来自反馈 #4）

访客 C 说首页"介绍的方面有点少"。我加了一个四宫格区块：架子鼓（10 级）、股票量化、游戏设计、阅读，每张卡片都链接到对应的页面（音乐页 / 量化项目详情 / 试玩页 / 知识库）。想法来源就是希望访客第一屏就知道"这个人是谁、玩什么"。



图 2-2 · 反馈 #4 前后对比：新增兴趣四宫格，卡片可点击跳转

2.4 技术路线流程图（V4 · 来自反馈 #3）

访客 B 看 Carbon Brain 项目的技术路线，希望"加个图示"。我用纯 CSS 做了五节点流程图：传感器原始数据 → 物理换算 → 18 维特征工程 → XGBoost 回归 → 饱和度估算。手机上箭头自动旋转 90° 变竖排。



图 2-3 · 反馈 #3 前后对比：技术路线从纯文字（左）变成流程图（右）

2.5 剧情游戏手机适配（V4 · 来自反馈 #6）

同学 djm 反馈：剧情游戏"用手机打开字迹大小会超出边框"。根因与修法见第三章 3.2。

2.6 更早的功能（V1–V3 简述）

- 真实后端联系表单（V2）：想法来源——"留言板必须真的能留言"，否则就是摆设；
- 知识库内容管线（V3）：Markdown 文件构建时自动解析成笔记页，加笔记只需加文件，不改代码；
- 页内反馈组件（V3）：想法来源——我想听到访客的真实声音，为 V4 埋下伏笔；
- 数字分身聊天入口（V3）：接入 AI 对话，访客可以和"数字版的我"聊天；
- 深浅双模式、动态效果、响应式布局（V3）：覆盖桌面 / 平板 / 手机三档。

三、遇到并解决的问题

这一章是我认为整个作业里最有含金量的部分：每个问题都先搞清楚为什么会发生，再决定怎么修，并给流程打补丁防止复发。

3.1 上线后知识库整页崩溃 (V3)

现象：知识库改造上线后，线上知识库页整页白屏崩溃，本地却一切正常。**原因：**重构时删除了一个 import，但漏删了两处对它的 watch 调用，组件 setup 阶段抛 ReferenceError。本地因为缓存没暴露，测试也因为"页面级测试盖不到组件 setup 的引用错误"而漏网。**解决：**10 分钟内定位修复并重新部署；随后新增 8 条导航测试（视口切换、返回栏分支、无效笔记 id、微信内置浏览器哨兵场景）堵住这类盲区，全量测试升至 106 绿。**教训：**测试要覆盖"渲染路径"而不只是"交互路径"。

3.2 剧情游戏在手机上文字超出边框 (V4 · 反馈 #6)

现象：同学用手机打开剧情游戏，文字溢出边框。**排查：**我先在本地用 320 / 360 / 390px 三档宽度实测游戏本体——竟然都不溢出。这说明不是布局 bug。**根因：**移动端浏览器（尤其微信内置浏览器）默认 `text-size-adjust: auto`，会自动放大 12px 以下的小字，导致部分浏览器里文字被强行放大撑破容器。**解决：**在游戏源文件加 `html{-webkit-text-size-adjust:100%;text-size-adjust:100%}` 禁止浏览器自作主张，重建部署后 iframe 加载场景也复验通过。**收获：**"在我机器上没问题"不等于没问题——真机反馈是测试仪器的延伸。

3.3 动效过度 (V4 · 反馈 #2)

首屏萤火虫、漂浮、滚动显现叠加在一起，有访客觉得"花"。除了删减（见 2.2），我还借此确立了动效三原则：**装饰动效不许抢内容、必须尊重 prefers-reduced-motion、悬停反馈只给桌面精确指针**。这比单纯"删特效"更值钱——以后每次加动效都有了验收标准。

3.4 与 AI 协作的流程事故 (V4 期间，三次翻车)

这轮迭代里 AI（DeepWorks + GLM 模型）承担了大量编码与文档工作，也翻过三次车，每次都值得记录：

- 文档污染事故：**AI 用全局文本替换填作业文档时，替换锚点不唯一，误伤了反馈记录表 7 行，而且命令没有用"&&"串联，检查脚本报错后 git 提交仍然执行了——脏版本被推上了远端。修复：从 git 历史恢复干净版本重写；流程补丁：所有"改文档→校验→提交"强制用 && 串联，校验不过就不许提交，替换锚点必须全文唯一并带断言。
- 超时误诊：**部署时推送迟迟无响应，AI 怀疑网络故障排查了 20 分钟，最后发现是 macOS 根本没有 `timeout` 命令——诊断工具本身不可用导致误判。教训：排障前先确认"工具在自己的环境里存在"。
- 不编造的底线：**有一次材料里只剩 1 条反馈，AI 没有为了好看而编造反馈数量，而是如实记录"当前仅收集到 1 条"，等真实反馈补齐后才更新为 6 条。这是我认为 AI 协作里最重要的品质：诚实优先于漂亮。

四、收集到的反馈与决策过程

反馈渠道：V3 上线的页内反馈组件（直达邮箱）+ 朋友圈扫码访问 + 同学直接反馈。共收集 6 条，全部真实、全部处理，包括不采纳的。

#	反馈（原文大意）	来源	分类	决策
1	内容多，不知道先看哪块	访客 A（站内组件）	内容	✅ P0 修复：亮点前置 + 分节目录跳转

2	动效有点花	访客 B	视觉	✅ P1: 与 #5 合并处理, 背景收敛
3	技术路线希望加图示	访客 B	内容	✅ P2: 五节点流程图 (手机竖排)
4	介绍的方面有点少	访客 C	内容	✅ P2: 兴趣四宫格 (可点击跳转)
5	图片展示可以加特效	访客 D	视觉	✅ P1: 与 #2 合并, 前景悬停增强
6a	游戏手机上文字超出边框	同学 djm	功能	✅ P0: text-size-adjust 修复
6b	触屏特效是什么意思🤔	同学 djm	交互	❌ 不改动: 那是更早版本已删除的轮播残留描述, 按"少即是多"原则保持删除 (详见下方讨论)

4.1 决策原则

- 影响可读性的排最前: #1 和 #6a 直接妨碍获取内容, 定为 P0 当轮就修;
- 方向相反的反馈合并思考: #2 (太花) 和 #5 (想加特效) 不冲突——收背景、放前景, 一收一放同时满足;
- 不采纳也要能说清理由: #6b 问的"触屏特效"是历史轮播功能的残留, 那个功能因为性能问题早已删除。重新加回来只为解释一句话, 代价 (代码复杂度 + 动效原则倒退) 远大于收益, 所以选择不恢复, 并在文档里写明原因。

4.2 验证闭环

每条修复都走了同一套流程: **改前截图** → **修复** → **部署** → **线上验证** → **回归测试**。图 2-1 ~ 2-3 就是"改进前/改进后"对比证据。回归测试按课件六项清单全部实测: 桌面布局、手机布局、导航按钮、图片链接、反馈表单 (真实提交了一封测试邮件并收到)、加载速度 (冷缓存首字节 102ms / 完整加载 481ms), **6/6 通过**。



图 4-1 · 回归测试之反馈表单：真实提交成功（邮件已送达邮箱留档）

五、简短反思

1. 最难的部分是什么？不是某个具体技术，而是决定“不做什么”：动效收敛到什么程度、触屏特效要不要恢复、数字统计区和四宫格会不会重复。每砍一个想法都在跟“反正能做”的诱惑作斗争。
2. 学到最重要的东西是什么？反馈驱动开发的完整闭环——先上线一个能收集反馈的版本（V3 的反馈组件），然后让真实访客决定下一步。这比任何“我觉得用户想要”都可靠。

3. 如果重来一次会怎么做？会更早把真机测试纳入流程（游戏溢出问题本地三档宽度全测过了，还是漏了真机浏览器行为）；会更早建立“改文档必须带唯一锚点断言”的纪律，省去一次污染事故。

4. AI 在这个项目里扮演了什么角色？结对程序员 + 文档员。它写代码、跑测试、部署、记录，我出方向、做决策、验证结果。三次翻车（3.4 节）让我明白：AI 的产出必须被流程约束（校验串联、锚点断言），但它的诚实值得信任——没收到反馈时它如实说 1 条，没有编成 6 条。

5. 这个项目对我意味着什么？第一次完整体验“发布 → 收集反馈 → 迭代 → 再发布”的产品循环。看到同学因为我的修复而“😓😓”变“😄”，比任何测试通过都让人满足。

附：更完整的材料随邮件附上——全部 AI 对话记录（00—09 十份）、AI 协作日志（案例 A—M）、测试与验收记录（114 条断言证据）、反馈决策表、回归测试清单与迭代反思，均按 V1—V4 版本分类整理。

线上站点持续更新于 <https://carrotsoup123456.github.io/homepage/>